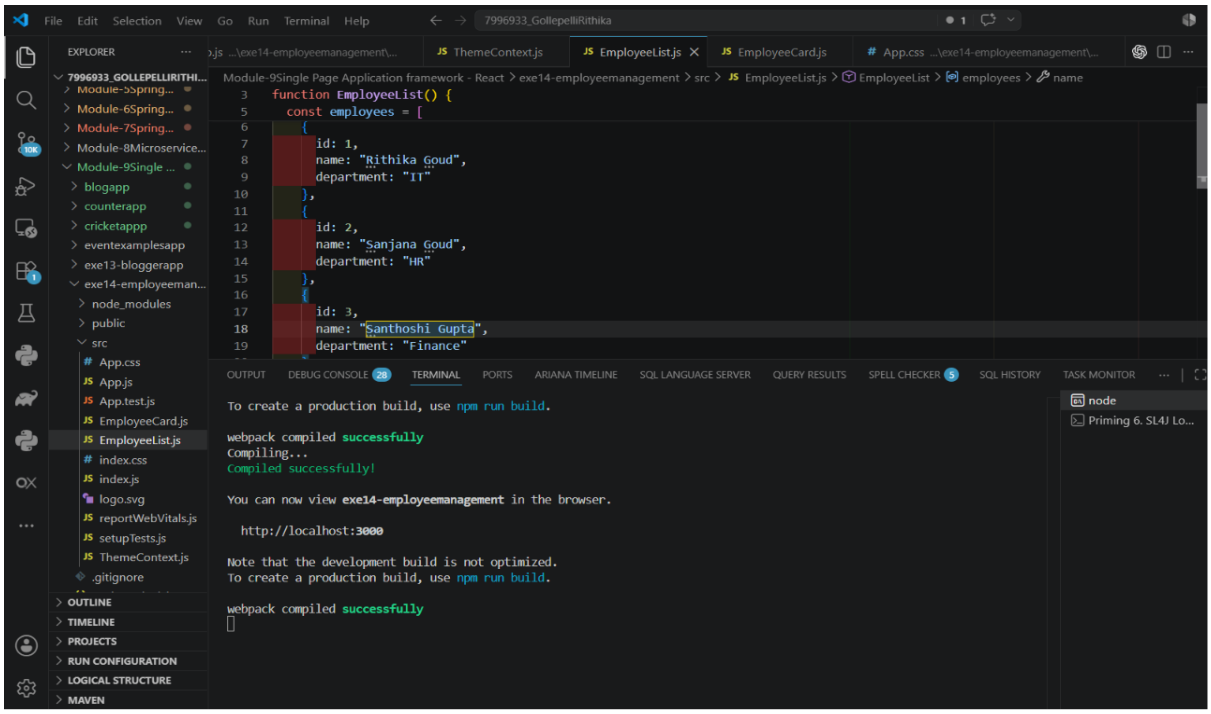
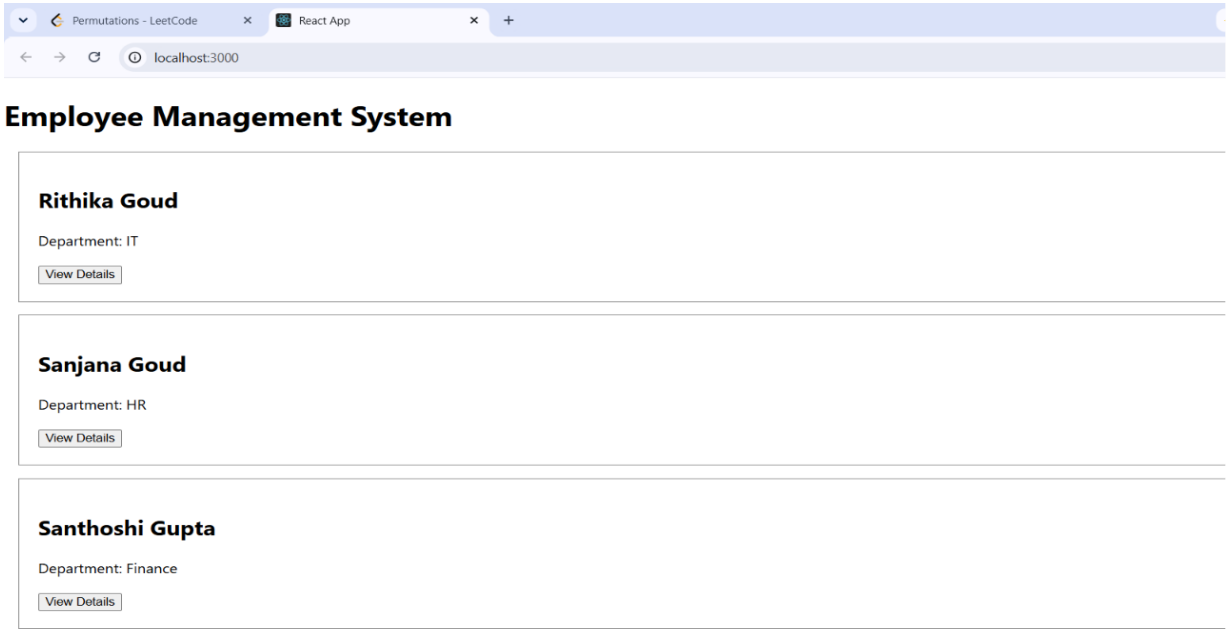
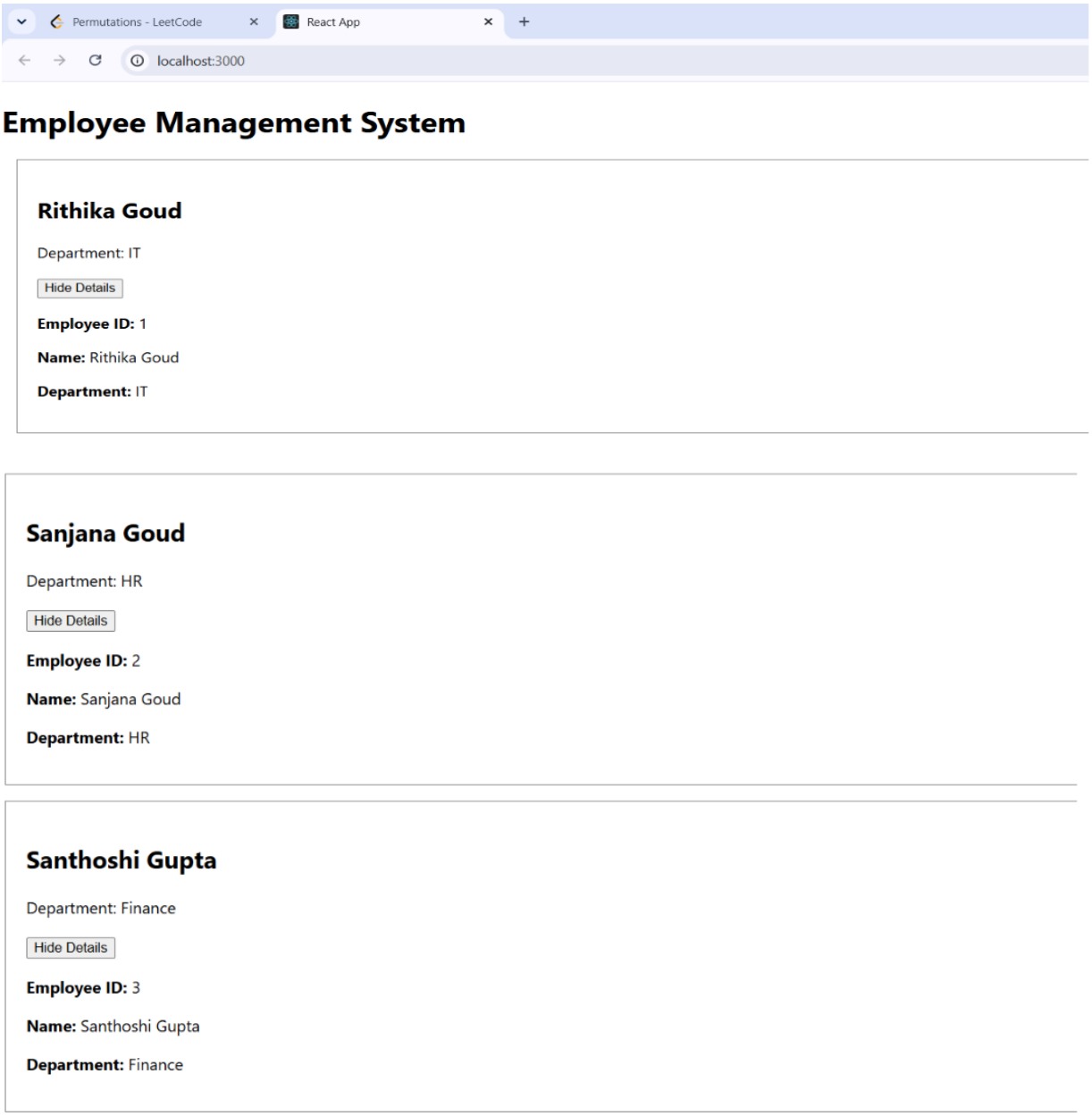
Output14exe14Employeemanagement.React.js-HOL



The Employee Management System application displays a list of employees with their names and departments. Each employee has a View Details button.



When the View Details button is clicked for Rithika Goud, the application updates the component's state using React's useState() hook. The button text changes from View Details to Hide Details, indicating that the employee's information is currently visible.



Objectives

1. Explain the Need and Benefits of React Context API

The React Context API is a built-in feature that allows data to be shared across multiple components without manually passing props through every intermediate component (a problem known as prop drilling). It provides a centralized way to manage and access common data such as user information, application themes, language preferences, or authentication status.

Need for React Context API

- Eliminates prop drilling.
- Makes data sharing between components easier.
- Simplifies state management for global data.

Benefits

- Shares data globally across components.
- Reduces unnecessary prop passing.
- Improves code organization.

2. Working with createContext()

createContext() is a React method used to create a Context object. Once the context is created, it provides a Provider component to supply data and a Consumer (or the useContext() Hook) to access that data in any child component.

Steps to Use createContext()

1. Create the context.
2. Wrap the required components with Context.Provider.
3. Pass data using the value prop.
4. Access the data using useContext().

Advantages

- Avoids passing props through multiple levels.
- Makes global data easily accessible.

3. List the Types of Router Components

React Router provides different router components to handle navigation based on the application's requirements.

Router Component	Description
BrowserRouter	Uses the browser's History API and is the most commonly used router for web applications.
HashRouter	Uses the URL hash (#) for routing and is useful for static hosting environments.
MemoryRouter	Stores navigation history in memory and is mainly used for testing or non-browser environments.
StaticRouter	Used for server-side rendering (SSR) applications.